Project 3902 Math

Saurav Poudyel

March 2025

Character Motion: Velocity Calculation

A character's motion in a two-dimensional plane can be represented by a velocity vector $\mathbf{v}$. Given an orientation angle θ, the velocity is computed as follows:

$$\mathbf{v} = v \begin{bmatrix} \sin(\theta) \\ -\cos(\theta) \end{bmatrix}$$

Derivation: This setup is common in game development to ensure intuitive direction handling:

- $\theta = 0$: Upward direction. - $\theta = \frac{\pi}{2}$: Rightward direction. - Ensures alignment with common screen coordinate systems.

Example Calculation: For a speed $v = 250$ and angle $\theta = \frac{\pi}{4}$ (45 degrees):

$$\mathbf{v} = 250 \begin{bmatrix} \sin(\frac{\pi}{4}) \\ -\cos(\frac{\pi}{4}) \end{bmatrix} = 250 \begin{bmatrix} 0.707 \\ -0.707 \end{bmatrix} = \begin{bmatrix} 176.8 \\ -176.8 \end{bmatrix}$$

Rotational Transformations: Bounding Box

The rotated bounding box is obtained by applying rotation matrices to corner points of an object's local coordinate space.

Corners in local space (centered at origin):

$$\mathbf{p}_1 = \begin{bmatrix} -\frac{w}{2} \\ -\frac{h}{2} \end{bmatrix}, \quad \mathbf{p}_2 = \begin{bmatrix} \frac{w}{2} \\ -\frac{h}{2} \end{bmatrix}, \quad \mathbf{p}_3 = \begin{bmatrix} \frac{w}{2} \\ \frac{h}{2} \end{bmatrix}, \quad \mathbf{p}_4 = \begin{bmatrix} -\frac{w}{2} \\ \frac{h}{2} \end{bmatrix}$$

Rotation by angle θ uses the matrix:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Thus, the rotated position for a corner point $\mathbf{p}_i$ is:

$$\mathbf{p}_i^{rot} = R(\theta)\mathbf{p}_i + \mathbf{p}_{\text{center}}$$

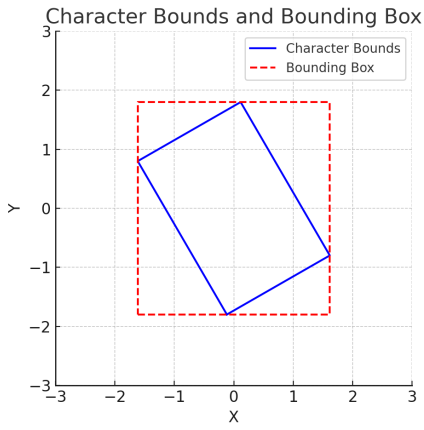

Figure 1: Rotated bounding box representation

Line of Sight for Mobs

In AI behavior, line-of-sight calculations help determine whether a mob can "see" the player. Raycasting is commonly used for this.

A common method follows the equation:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \geq 0$$

where:

- $\mathbf{o}$ is the mob's position. - $\mathbf{d}$ is the direction to the target.

For implementation in MonoGame, refer to: Line of Sight with Ray in XNA/MonoGame.

Circular and Rectangular Collision

Collision detection is an essential aspect of physics simulation in games. Two common types:

- **Circular Collision**: Check distance between centers. - **Rectangular Collision**: Axis-Aligned Bounding Box (AABB) intersection.

A rectangle-rectangle collision check is computed via:

$$(x_{min1} < x_{max2} \land x_{max1} > x_{min2}) \land (y_{min1} < y_{max2} \land y_{max1} > y_{min2})$$

For further details on implementations in MonoGame: 2D Collision Detection in MonoGame.

Raycasting for Obstacle Detection

Raycasting traces a line (ray) from an origin point in a given direction to detect obstacles. The equation:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \geq 0$$

where:

- $\mathbf{o}$ is the starting position. - $\mathbf{d}$ is the normalized direction vector.

For performance optimization in MonoGame: 2D Raycasting Performance in MonoGame.

Reflection Vector

The reflection of an incident vector $\mathbf{i}$ upon a surface normal $\mathbf{n}$ is:

$$\mathbf{r} = \mathbf{i} - 2(\mathbf{i} \cdot \mathbf{n})\mathbf{n}$$

Example Calculation: Incident vector $\mathbf{i} = [1, -1]$ and normal vector $\mathbf{n} = [0, 1]$:

$$\mathbf{r} = [1, -1] - 2\left(([1, -1] \cdot [0, 1])\right)[0, 1] = [1, -1] - 2[-1][0, 1] = [1, 1]$$

This indicates reflection off a horizontal surface.